

Atividade Avaliativa — Design Patterns em Python

Sistema de pedidos

Uma loja vende pelos canais WEB e MOBILE. O sistema registra pedidos, processa o pagamento, apresenta o checkout do canal e envia a notificação correspondente. A configuração da aplicação é compartilhada.

Desenvolvam a solução em Python usando Singleton, Builder, Factory Method e Abstract Factory. A divisão das responsabilidades e a inclusão posterior de um canal serão usadas para avaliar, respectivamente, SRP e OCP.

Ao longo da atividade, novas alterações serão solicitadas ao sistema. A solução deverá ser estruturada de modo que essas alterações não exijam modificações desnecessárias em componentes que não são responsáveis pelo novo comportamento.

1. Configuração da aplicação

Criem AppConfig com `environment = "production"`, `currency = "BRL"` e `debug = False`. Usem `__new__` para que duas chamadas a AppConfig() devolvam o mesmo objeto. Uma chamada posterior não pode restaurar os valores iniciais de atributos já alterados.

Demonstrem, por meio de testes:

- que duas referências obtidas para a configuração representam o mesmo objeto;
- que uma alteração realizada em uma referência pode ser observada pela outra;
- que os valores alterados não são restaurados quando a configuração é obtida novamente.

Resposta escrita

1. Explique por que utilizar `__new__` não impede, por si só, novas execuções de `__init__`.
2. Explique como um módulo Python poderia ser utilizado para compartilhar uma configuração sem implementar uma segunda versão do Singleton.
3. Identifique uma possível consequência de possuir um objeto de configuração global compartilhado.

Não é necessário implementar uma segunda solução de Singleton.

2. Pedido e Builder

Criem Order e OrderBuilder. Todo pedido precisa ter cliente; pode conter vários produtos e, opcionalmente, endereço, cupom, forma de pagamento escolhida e observação. Cada produto tem nome e preço. Order.total() devolve a soma dos preços; não é preciso aplicar descontos.

Os métodos do builder devem permitir encadeamento. build() deve rejeitar a construção sem cliente.

Demonstrem, por meio de testes:

- a construção de um pedido contendo pelo menos dois produtos;
- a utilização de pelo menos dois atributos opcionais;
- a tentativa de construir um pedido sem cliente.

Resposta escrita

1. Identifique quais componentes da sua implementação correspondem ao Builder e ao objeto construído.

2. Explique por que seria possível construir o pedido diretamente pelo construtor de Order e qual seria a diferença em relação à solução adotada.

3. Pagamento e Factory Method

Definam Payment com pay(amount) e implementem PixPayment, CreditCardPayment e BoletoPayment. PaymentProcessor deve conter o fluxo process_order(order): criar o pagamento por meio de create_payment() e chamar pay(order.total()). Implementem create_payment() em PixProcessor, CreditCardProcessor e BoletoProcessor.

O fluxo comum fica em PaymentProcessor e não deve instanciar diretamente uma classe concreta de pagamento.

Demonstrem, por meio de testes:

- O processamento de pedidos utilizando pelo menos duas formas de pagamento diferentes.
- Que a forma de pagamento registrada no pedido corresponde ao mecanismo utilizado no processamento.

Resposta escrita

Com base na implementação desenvolvida:

1. Identifique os papéis de: Creator, Concrete Creator, Product, Concrete Product.
2. Explique por que uma função contendo simplesmente uma sequência de if/elif escolhendo classes concretas não é, por si só, suficiente para caracterizar o padrão Factory Method.
3. Considere que uma nova forma de pagamento seja adicionada posteriormente. Explique quais partes da sua implementação precisariam ser alteradas.

4. Famílias por canal

Canal de venda e forma de pagamento são escolhas independentes: WEB pode usar PIX e MOBILE pode usar cartão. A fábrica do canal não cria objetos Payment. Definam Checkout com show(order), Notification com send(order) e ChannelFactory com create_checkout() e create_notification(). WebFactory cria WebCheckout e WebNotification; MobileFactory cria MobileCheckout e MobileNotification. O código que usa a fábrica deve trabalhar com as abstrações.

Resposta escrita

Com base na implementação desenvolvida:

1. Explique por que checkout e notificação podem ser considerados uma família de produtos.
2. Explique qual problema a Abstract Factory resolve nessa situação.
3. Explique por que o pagamento não deve fazer parte da fábrica responsável pelo canal.

5. Seleção de fábrica e alteração do sistema

Implementem get_channel_factory(channel) com um registro de fábricas. Um canal desconhecido deve produzir um erro claro. Inicialmente, registrem WEB e MOBILE.

Em seguida, acrescentem KIOSK, com KioskCheckout, KioskNotification e KioskFactory. Registrem a nova fábrica sem editar get_channel_factory nem o código que usa Checkout e Notification

Resposta escrita

1. Liste os arquivos criados ou alterados para adicionar KIOSK.
 2. Explique por que as alterações realizadas são ou não compatíveis com o princípio OCP.
-

6. Responsabilidades e integração

Uma primeira versão de OrderService reunia create_order, calculate_total, create_payment, send_notification e save_log. Identifiquem as responsabilidades misturadas e distribuam essas operações entre os componentes da solução. A classe que coordena o fluxo pode chamar colaboradores; ela não deve construir pedidos, escolher classes concretas de pagamento ou notificação, nem armazenar a configuração global. Se houver registro de eventos, atribua essa função a um componente próprio.

Apresentem uma execução completa: obtenção de AppConfig, construção do pedido, escolha do canal, apresentação do checkout, processamento do pagamento e envio da notificação. Repitam o fluxo com KIOSK. A escolha inicial de um PaymentProcessor e o registro das fábricas podem ser feitos na inicialização da aplicação.

Resposta escrita

Responda considerando **a implementação efetivamente entregue pela dupla**:

1. Qual é a responsabilidade principal de cada componente criado?
 2. Escolha três componentes diferentes e indique uma mudança que deveria ficar restrita a cada um deles.
 3. Identifique uma decisão de projeto da sua solução que poderia ser diferente. Explique qual seria a alternativa e qual seria a consequência dessa mudança.
-

7. Testes e alterações

Além dos testes solicitados nas questões anteriores, a dupla deverá criar **três testes adicionais**, sendo:

- um teste relacionado ao Singleton;
- um teste relacionado ao Builder;
- um teste relacionado à criação de objetos por uma das fábricas.

Os três testes devem verificar comportamentos que não sejam exatamente os exemplos apresentados no enunciado.

Para cada teste, informe brevemente:

- o comportamento que está sendo verificado;
 - o resultado esperado;
 - por que esse comportamento é importante para a solução.
-

8. Situação de mudança

Considere agora que a empresa deseja adicionar uma **nova forma de pagamento** ao sistema, sem modificar o fluxo geral de processamento de pedidos.

A dupla deverá escolher uma nova forma de pagamento que não esteja entre as três inicialmente implementadas.

Implemente essa extensão.

Depois da implementação, responda:

1. Quais arquivos foram criados ou modificados?
2. O fluxo principal de processamento precisou ser alterado?
3. Quais classes existentes precisaram ser modificadas?
4. Explique como o Factory Method contribuiu para essa extensão.
5. Compare essa alteração com a inclusão do canal KIOSK. Quais são as semelhanças e diferenças arquiteturais entre as duas extensões?

Entrega

A atividade é em dupla. Enviem a luishbuenopro@gmail.com, até as 23:59h do dia 23/09/2026, um arquivo compactado com os módulos Python, os testes e um exemplo executável da integração. Acrescentem um PDF separado com as respostas escritas solicitadas. A estrutura dos módulos fica a critério da dupla; expliquem a escolha no PDF.

Critérios de avaliação

Serão avaliados:

- funcionamento do sistema e dos testes;
- aplicação correta dos quatro Design Patterns;
- capacidade de justificar as decisões de projeto;
- separação de responsabilidades;
- aplicação de SRP e OCP;
- capacidade de estender a solução para novos requisitos;
- qualidade e abrangência dos testes;
- coerência entre o código entregue e as respostas do relatório.